



myP Script Functions Manual

Version 1.4.4

(corresponds to the myP version)

2020-03-10

The information contained in this document is property of F&P Robotics AG and shall not be reproduced in whole or in part without prior written approval of F&P Robotics AG. The information provided herein is subject to change without notice and should not be constructed as a commitment by F&P Robotics AG. This manual is periodically reviewed and revised. F&P Robotics AG assumes no responsibility for any errors or omissions in this document.

This manual consists of original instructions from F&P Robotics AG and is intended for all users of myP. Before using the functions mentioned in this document together with myP and a P-Rob robotic arm or P-Grip robotic gripper, you must read the corresponding user manuals carefully, particularly paying attention to the safety instructions on the P- Rob user manual. Compliance with the instructions in all manuals is mandatory.

Copyright © 2011-2020 by F&P Robotics AG. All rights reserved.

The F&P logo as well as P-Rob®, P-Grip® and myP® are registered trademarks of F&P Robotics AG.

Contents

Basic Python Programming	6
Basic Python Standards	6
Commentary	6
Variable Definitions	6
Variable Conversions	6
Data Structures	7
Basic Python Functions	8
Standard and Default Function Arguments	8
Unpacked Function Arguments	8
Basic Python Operators	9
Calculation Operators	9
Comparison Operators	9
Logical Operators	9
Bitwise Operators	9
Conditional Programming	10
If–Else Condition	10
While Loop	11
For Loop	12
myP Core Module Functions	13
Robot Movements	13
Move Joint	13
Move Tool	15
Move to Pose	17
Play Path	18
Run Simple Path	19

Wait for Motor	21
Stop Motion	21
Is Motor Moving	21
Kinematic Functions	23
Read TCP Pose	23
Get Grid Points	23
Forward Kinematics	24
Inverse Kinematics	25
Get Pose Angles	26
Get Pose Data	26
Read IMU Data	26
Program Flow Scheduling	27
Wait	27
Run Script	27
Start Parallel Script	27
Stop Application	28
Pause Application	29
Resume Application	29
Stop and Release	29
Is Status Ready	30
Is Status Running	30
Is Status Paused	30
Is Status Busy	30
User Communication	31
Dialog Functions	31
Print	33
Say	33

Create TCP Client	35
Create TCP Server	35
Create UDP Socket	36
Send Message	36
Receive Message	36
Clear Messages	37
Socket	38
Socket Receive	38
Socket Send	38
Socket Close	38
Custom Error Handling	39
Raise Error	39
Raise Warning	39
Log Message	39
Clear Log	39
Application Variable Functions	41
Set Variable	41
Get Variable	41
Remove Variable	42
Digital I/O Functions	43
Read Digital Inputs/Outputs	43
Write Digital Outputs	44
Reading Motor Data	45
Read Actuator Position/Velocity/Current	45
Calibration Functions	46
Default Calibration	46
Calibrate Joint	46

Collision Detection	48
Get Collision Sensitivity	48
Set Collision Sensitivity	48
Get Collision Mode	49
Set Collision Mode	49
P-Grip Module Functions	50
Open Gripper	50
Close Gripper	50
Read Gripper Angle	51
Calibrate Gripper	51
Get Sensors	52
Read Sensor Data	52
Linear Axis Module Functions	53
Move Linear Axis	53
Calibrate Linear Axis	54
ROS Handler Functions	55
Publish to ROS Topic	55
Subscribe to ROS Topic	55
Read ROS Topic	56
Unregister from ROS Topic	57
Wait for ROS Topic	57
Get Available ROS Services	57

1 Basic Python Programming

This section explains some of the basic [Python](#) programming that can be used in the application editor of myP. This is merely a summary of the most important Python (3.5) standards, of course there are many more Python built-in functions available. Please keep in mind that some functionalities of Python are restricted in myP though. For advanced Python users or programming tutorials, we hereby refer to the main online [Python Documentation](#).

1.1 Basic Python Standards

Commentary

The following code lines represent code comments which are not executed on run-time.

```
# this is a commentary and will not be considered as executable code
"""
This is a multiline commentary
that will also not be executed.
"""
```

Variable Definitions

Python supports many different variable types and does not differentiate between variable definition and variable declaration.

```
my_boolean = True           # defining "my_boolean" as a Boolean with value True
my_integer = 3              # defining "my_integer" as an integer number with value 3
my_float = 3.1416           # defining "my_float" as a floating-point number 3.1416
my_string = "Hello World"  # defining "my_string" as a string with value "Hello World"
```

Variable Conversions

Variables can easily be converted from one type to another, if the values are compatible.

```
x = bool(x)                # converts x to a boolean
x = int(x)                  # converts x to an integer number
x = float(x)                # converts x to a floating-point number
x = str(x)                  # converts x to a string
```

Data Structures

Python supports four different data structure types:

- A **List** contains an ordered array of elements.

```
my_list = list()           # create an empty list
my_list = [1, 2, 3, 4, 5]  # create a list containing some numbers
my_list.append(6)          # add element to existing list
my_list[0] = 6             # sets the first list element to be 7
```

- A **Tuple** contains an ordered but immutable (unchangeable) array of elements.

```
my_tuple = tuple()        # create an empty tuple
my_tuple = (1, 2, 3, 4, 5) # create a tuple containing some numbers
```

- A **Set** contains an unordered array of elements that contains no duplicates.

```
my_set = set()            # create an empty set
my_set = {1, 2, 3, 4, 5}  # create a set containing some numbers
my_set.intersection({1, 3, 5, 7, 9}) # intersection of two sets
my_set.union({1, 3, 5, 7, 9})        # union of two sets
```

- A **Dictionary** contains an unordered combination of keys and values.

```
my_dictionary = dict()      # create an empty dictionary
my_dictionary = {"a": 1, "b": 2} # create a dictionary containing key-value pairs
my_dictionary["a"] = 3       # sets the element "a" to be 3
```


1.2 Basic Python Functions

Standard and Default Function Arguments

Let's consider that a function has been defined with the following arguments:

```
function(arg1, arg2, arg3="x", arg4="y")
```

The function arguments **arg1** and **arg2** are standard arguments that need be given when calling this function. The arguments **arg3** and **arg4** have a default value and therefore do not necessarily be given when calling this function. All the following function calls are valid:

```
function("a", "b")                # -> function("a", "b", "x", "y")
function("a", "b", "c")           # -> function("a", "b", "c", "y")
function("a", "b", "c", "d")      # -> function("a", "b", "c", "d")
function("a", "b", arg4="d")      # -> function("a", "b", "x", "d")
function(arg1="a", arg2="b", arg3="c", arg4="d") # -> function("a", "b", "c", "d")
function(arg3="c", arg2="b", arg1="a") # -> function("a", "b", "c", "y")
```

Unpacked Function Arguments

A few myP functions use unpacked arguments. These arguments are useful if the number of arguments of a function may vary. Let's consider the following functions arguments:

```
function1(*arguments)           # function with unpacked list argument
function2(**arguments)          # function with unpacked dictionary argument
```

Then these functions can be used in the following way:

```
function1(value1, value2, ...)   # function with unpacked list argument
function2(arg1=value1, arg2=value2, ...) # function with unpacked dict argument
```

1.3 Basic Python Operators

Calculation Operators

```
x = 3 + 5      # addition:      the value of x is now 8
x = 7 - 2      # subtraction:   the value of x is now 5
x = 4 * 8      # multiplication: the value of x is now 32
x = 54 / 6     # division:      the value of x is now 9
x = 16 % 7     # modulo:        the value of x is now 2
x = 2 ** 3     # exponential:   the value of x is now 8
```

Comparison Operators

Here x and y can be of any type.

```
x == y        # returns true if x is equal to y
x != y        # returns true if x is not equal to y
```

Here x and y are numbers.

```
x > y         # returns true if x is greater than y
x < y         # returns true if x is smaller than y
x >= y        # returns true if x is greater than or equal to y
x <= y        # returns true if x is smaller than or equal to y
```

Logical Operators

Here x and y are Booleans.

```
x and y       # logical AND returns true if x and y are true
x or y        # logical OR returns true if either x or y is true
not x         # logical NOT returns true if x is false
```

Bitwise Operators

Here x and y are integers.

```
x & y          # bitwise AND returns an integer resulting
               # from bitwise logical AND of x and y
x | y          # bitwise OR returns an integer resulting
               # from bitwise logical OR of x and y
```

1.4 Conditional Programming

If-Else Condition

"if", "elif" (else if) and "else" conditions can be combined according to the following rules:

- The "if" scope is executed if and only if its condition returns the value True.
- An "elif" (else if) scope is optional and executed if and only if all previous "if" or "elif" conditions returned the value False and its own condition returns the value True.
- An "else" scope is optional and executed if and only if all previous "if" or "elif" conditions returned the value False.

The general if-elif-else code looks as follows:

```
if condition_1:
    # the following code is executed if 'condition_1' is True
    ...
elif condition_2:
    # the following code is executed if 'condition_1' is False
    # and 'condition_2' is True
    ...
elif condition_3:
    # the following code is executed if 'condition_1' and 'condition_2' are False
    # and 'condition_3' is True
    ...
else:
    # the following code is executed if all previous conditions leading up
    # to this 'else' statement are False
    ...
```

Example

Checking if a variable "my_integer" is equal to 5, 10, or neither one of those numbers.

```
if my_integer == 5:
    print("your integer is 5.")
    print("have a nice day.")
elif my_integer == 10:
    print("your integer is 10.")
    print("have a nice day.")
else:
    print("your integer is neither 5 nor 10.")
    print("have a nice day anyway.")
```

While Loop

A while loop is executed and repeated according to the following rules:

- The "while" loop scope is executed repeatedly as long as its condition returns the value True.
- A "continue" keyword is optional and can be used to jump to the next loop iteration ignoring the remaining code within the while loop.
- A "break" keyword is optional and can be used to jump out of this loop.
- An "else" scope is optional (and rarely used) and executed if the loop condition turns to True. In other words, this scope is not executed if the while loop exits with a break statement.

```
while condition:
    # the following code is executed repeatedly as long as 'condition' is True
    ...
    if condition_1:
        # the 'continue' keyword skips the remaining code of this loop
        # and jump to the next iteration
        continue
    if condition_2:
        # the 'break' keyword breaks out of this loop, no matter the loop condition
        break
else:
    # the following code is executed only if the while loop exits
    # by setting 'condition' to False
    ...
```

Example

Printing all numbers from 1 to 10.

```
index = 1
while index <= 10:
    print(index)
    index += 1
```

Example

Infinite Loop: Of special interest to a lot of applications are infinite loops, in order for P-Rob to perform a task repetitively until it is stopped manually using the appropriate myP functions. If using infinite loops, make sure that myP does not iterate through this loop too fast, otherwise myP might not be fast enough to process other functions (like sending a stop command to the robot) in parallel to executing the application.

```
import time
while True:
    ...
    time.sleep(0.1)
```

For Loop

A for loop is executed and repeated according to the following rules:

- The "for" loop scope is executed repeatedly, once for each item in an iterable sequence (e.g. a list, a set, a dictionary, or a string).
- A "continue" keyword is optional and can be used to jump to the next loop iteration ignoring the remaining code within the for loop.
- A "break" keyword is optional and can be used to jump out of this loop.
- An "else" scope is optional (and rarely used) and executed after the last item in the for loop sequence has been processed. In other words, this scope is not executed if the for loop exits with a brake statement.

```
for item in sequence:
    # the following code is executed once for each 'item' in 'sequence'
    ...
    if condition_1:
        # the 'continue' keyword skips the remaining code of this loop
        # and jump to the next iteration
        continue
    if condition_2:
        # the 'break' keyword breaks out of this loop, no matter the loop condition
        break
else:
    # the following code is executed only if the for loop exits
    # by finding no more 'item' in 'sequence'.
    ...
```

Example

Printing all numbers from 1 to 5.

```
for index in range(5):
    print(index)
```

Example

Printing my favorite fruits.

```
for fruit in ["apple", "banana", "orange"]:
    print("one of my favorite fruits is " + str(fruit))
```

Example

Counting out the letters in "python".

```
index = 1
for letter in "python":
    print("letter number " + str(index) + " in 'python' is " + str(letter))
    index += 1
```

2 myP Core Module Functions

2.1 Robot Movements

Move Joint

```
move_joint(actuator_ids, position, velocity=None, acceleration=None,  
           block=True, relative=False, mode="trapezoidal")
```

This function moves a single joint, or multiple joints of the robot to a specific position or set of positions in the joint space, respectively. This function has been designed such that the movement of each joint finishes exactly at the same time.

Parameter	Types	Description
actuator_ids	Various	This argument contains the IDs of the motors to move. It can be - the ID of a single joint (e.g. 6), - a list of joint IDs for multiple joints (e.g. [1,4,6]), - the global constant ALL_KIN_JOINTS, which is equal to the list of all kinematic joints, - the global constant ALL_ACTUATORS, which is equal to the list of all actuators (e.g. including end-effectors).
position	Float or List of Floats	This argument contains the angles (in degrees) that are assigned to the joints. It can be - a single floating-point number that will be assigned to each joint (e.g. -30), - a list of floating-point numbers that will be assigned to each joint specified in actuator_ids (e.g. [90, -45, 30]).
velocity	Float or List of Floats	This argument contains the angular velocity (in degrees per second) that is used during the motion. It can be - a single floating-point number that defines the maximum velocity of each joint (e.g. 45), - a list of floating-point numbers that will be assigned to each of the joints specified in actuator_ids (e.g. [45, 25, 30]). Please note that if you are using this option, the movements of the joints will not necessarily finish at the same time, - or any of the floating-point numbers can be replaced by "MAX" to move with maximum possible velocity. If no argument is given, a relatively slow default velocity is chosen.
acceleration	Float or List of Floats	This argument contains the angular acceleration (in degrees per second squared) that are assigned to the joints. It can be - a single floating-point number that defines the maximum acceleration of each joint (e.g. 60), - a list of floating-point numbers that will be assigned to each of the joints specified in actuator_ids (e.g. [60, 35, 45]). Please note that if you are using this option, the movements of the joints will not necessarily finish at the same time,

		- or any of the floating-point numbers can be replaced by "MAX" to move with maximum possible acceleration. If no argument is given, the acceleration is chosen automatically.
block	Boolean	If True , the next command will be executed once this motion has finished. If False , the next command will be executed without waiting for this motion to finish.
relative	Boolean	If True , the position values are interpreted as a relative offset to the current joint angles. If False , the position values are interpreted as absolute joint angles.
mode	String	This argument specifies the motion mode, which refers to the shape of the executed velocity profile. - The "trapezoidal" velocity profile produces time optimal joint trajectories and induces a piecewise constant acceleration profile without jerk limitations. - The "s-curve" mode produces a smoother motion, though it is slightly slower, requires the initial velocities to be zero and the blocking flag to be set to True.

Examples

```
move_joint(ALL_KIN_JOINTS, 0, mode="s-curve")
# ... this is similar to ...
move_joint([1, 2, 3, 4, 5, 6], [0, 0, 0, 0, 0, 0],
          None, None, True, False, "s-curve")
```

Moving all kinematic joints of the robot with one smooth motion to 0°, which is executed in a calibrated robot causes it to stand up straight.

```
move_joint(6, -30, 45, "MAX", relative=True)
```

Moving joint 6 by -30° from its current position, with a velocity of 45°/s and acceleration of 60°/s².

```
move_joint([2,3,5], [20,-40,20], 35, 50)
```

Moving joints 2, 3 and 5 to 20°, -40° and 20°, respectively. The motion will be performed such that each joint will finish its movement at the same time. The joint that has the biggest angular distance to move, will move with a velocity of 35°/s and acceleration of 50°/s².

```
move_joint([1,4,6], [90, -45, 30], [45, 25, "MAX"], [60, 35, "MAX"], block=True)
# ... this is similar to ...
move_joint(1, 90, 45, 60, block=False)
move_joint(4, -45, 25, 35, block=False)
move_joint(6, 30, "MAX", "MAX", block=True)
```

Simultaneously moving three joints that will not finish their movements at the same point in time due to specifically chosen velocity and acceleration values for each moving joint. While for joint 1 and 4 specific velocity and acceleration values are given, joint 6 uses its maximum possible values for reaching its target angle in the shortest possible duration.

Move Tool

```
move_tool(x, y, z, orientation=None, velocity=None, acceleration=None,
          block=True, relative=False, frame="base", mode="trapezoidal")
```

This function moves the tool center point (TCP) of the robotic arm to a specific target position. Optionally, the target orientation and the motion velocity and acceleration can be chosen. Also, the coordinate frame can be chosen in which to move the robot. For details about the mathematical conventions and the coordinate systems available in myP, please refer to the [myP User Manual](#).

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
x, y, z	Floats	These values define the three-dimensional position of the TCP (in mm) expressed in Cartesian coordinates in the world frame, the main coordinate system of the robot.
orientation	List of Floats	These values define the tool orientation (in degrees). The orientation is defined as [roll, pitch, yaw] .
velocity	Float	This argument contains the maximum angular velocity (in degrees per second) that is used during this motion (e.g. 45). Also, "MAX" can be used instead to move with maximum possible velocity. If no argument is given, a relatively slow default velocity is chosen.
acceleration	Float	This argument contains the maximum angular acceleration (in degrees per second squared) that is used during this motion (e.g. 60). Also, "MAX" can be used instead to move with maximum possible acceleration.
block	Boolean	If True , the next command will be executed once this motion has finished. If False , the next command will be executed without waiting for this motion to finish.
relative	Boolean	If True , the pose values of the TCP are interpreted as relative offsets to the current pose components. If False , the pose values are interpreted as absolute values. By default, the robot will move absolute if it moves in base coordinates and relative if it moves in tool coordinates.
frame	String	This value specifies the coordinate frame in which the robot will move. Allowed values are "base" and "tool" , to move in the robot base coordinates or the end-effector tool coordinates, respectively. By default, the "base" coordinates are chosen.
mode	String	This argument specifies the motion mode, which refers to the shape of the executed velocity profile. - The "trapezoidal" velocity profile produces time optimal joint trajectories and induces a piecewise constant acceleration profile without jerk limitations. - The "s-curve" mode produces a smoother motion, though it is slightly slower, requires the initial velocities to be zero and the blocking flag to be set to True.

Examples

```
move_tool(200, 600, 350, [30, -20, -90], mode="s-curve")
# ... this is similar to ...
move_tool(200, 600, 350, [30, -20, -90],
          relative=False, frame="base", mode="s-curve")
```

Moving the TCP to the position [200mm, 600mm, 350mm] in robot base coordinates, with orientation angles 30°, -20° and -90° for the roll, pitch and yaw angles, respectively.

```
move_tool(800, 0, 300, None, 25, "MAX")
```

Moving the TCP to the position [800mm, 0mm, 300mm] with a velocity of 25°/s and with maximum possible acceleration. The orientation is not given and will therefore be chosen by the function.

```
move_tool(20, 0, 0, relative=True, frame="tool")
# ... this is similar to ...
move_tool(20, 0, 0, frame="tool")
```

Moving the TCP by 20mm in the positive x-direction in tool coordinates (forward in direction of the gripper fingers). Please note, that the orientation will change during this motion. Use the additional "orientation = [0, 0, 0]" argument to prevent relative orientation change in all dimensions.

Move to Pose

```
move_to_pose(pose_name, velocity=None, acceleration=None,  
             block=True, offset=None, mode="trapezoidal")
```

This function moves the robot to a previously saved pose.

Parameter	Types	Description
pose_name	String	This argument contains the name of the pose to move to (e.g. "myPose" , if this name is compatible with the name of a previously saved pose).
velocity	Float	This argument contains the maximum angular velocity (in degrees per second) that is used during this motion (e.g. 45). Also, "MAX" can be used instead to move with maximum possible velocity. If no argument is given, a relatively slow default velocity is chosen.
acceleration	Float	This argument contains the maximum angular acceleration (in degrees per second squared) that is used during this motion (e.g. 60). Also, "MAX" can be used instead to move with maximum possible acceleration.
block	Boolean	If True , the next command will be executed once this motion has finished. If False , the next command will be executed without waiting for this motion to finish.
offset	List of Floats and String	This argument defines a position offset that is added to the target pose. The offset is given in the format [x, y, z, frame], where the values x , y , z are floating point numbers and frame specifies the coordinate frame in which the robot will move ("base" or "tool" , check out the description of the function "move_tool" for more details).
mode	String	This argument specifies the motion mode, which refers to the shape of the executed velocity profile. - The "trapezoidal" velocity profile produces time optimal joint trajectories and induces a piecewise constant acceleration profile without jerk limitations. - The "s-curve" mode produces a smoother motion, though it is slightly slower, requires the initial velocities to be zero and the blocking flag to be set to True.

Examples

```
move_to_pose("startPose", acceleration="MAX", mode="s-curve")
```

Moving to a pose named "startPose" with default velocity and maximum possible acceleration using a smooth S-Curve shaped velocity profile.

```
move_to_pose("endPose", 45, 60, block=False)
```

Moving to a pose named "endPose" with a maximum velocity of 45°/s and maximum acceleration of 60°/s² for each joint and awaiting new commands while moving.

Play Path

```
play_path(path_name, velocity=None, acceleration=None)
```

This function plays a previously created or recorded path. For details about creating a path, please refer to the path editor section of the [myP User Manual](#). The path can be either a "simple path" (using interpolated movements in joint space and online calculation) or an "advanced path" (using cubic spline, linear and/or circular interpolations, its trajectory need to be pre-calculated offline). Therefore, the additional velocity and acceleration input arguments can only be used for a simple path. To change the velocity and acceleration of an advanced path, the path editor should be used instead. Note that to immediately execute a path, the robot needs to be located at the very first pose given in the path before executing "play_path". Otherwise the robot will first move to the initial path pose using its (relatively slow) default velocity and acceleration values.

Parameter	Types	Description
path_name	String	This argument contains the name of the path to play (e.g. "myPath" , if this name is compatible with the name of a previously saved path).
velocity	Float or List of Floats	The velocity of a simple path is either a single floating-point number or a list of numbers (in degrees per second). If a single value is given, this value is used as the maximum velocity for all path segments. If a list is entered, the given velocity values correspond to the maximum velocity in each segment of the path.
acceleration	Float or List of Floats	The acceleration of a simple path can be given as a single number or a list of numbers (in degrees per second squared). If a single acceleration is given, this value is used as the maximum acceleration for all path segments. If a list is entered, the given acceleration values correspond to the maximum acceleration in each segment of the path.

Examples

```
play_path("myPath")
```

This example plays a path previously saved as "myPath", which can be either a simple or advanced path. Since velocity and acceleration values are not given specifically, therefore the path will execute using the speed values it was saved with.

```
play_path("mySimplePath", velocity=20, acceleration=25)
```

This example plays a simple path with the desired maximum velocity and acceleration values of 20°/s and 25°/s², respectively.

```
play_path("mySimplePath", velocity=[20, 10, 15], acceleration=[10, 50, 30])
```

This example plays a simple path with different velocity and acceleration values for each path segment. The number of elements in the velocity and acceleration arrays must be equal to the number of path segments. The path "mySimplePath" for example contains a list of four poses and therefore has three segments between consecutive poses.

Run Simple Path

```
run_simple_path(path_poses, velocity=None, acceleration=None)
```

This function runs a "simple path" (which is a path that uses interpolated movements in joint space) and which is calculated online before its execution. If no velocity or acceleration is passed as argument, standard default values will be chosen. Note that to execute a path quickly, the robot needs to be located at the very first pose given in the path before executing "run_simple_path". Otherwise the robot will first move to the initial path position using the (relatively slow) default velocity and acceleration values before running the path.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
path_poses	List of Poses	Contains all the poses of the path. The poses can be in the following formats: - name of pose saved in the database (e.g. "myPose") - actuator angles as list with values in degrees (e.g. [-10, 30, 30, -20, 45, 90]) - actuator angles as dictionary with values in degrees (e.g. {"1": -10, "2": 30, "3": 30, "4": -20, ...}) - list of [x, y, z, orientation] with values in mm and degrees (e.g. [600, 0, 300, [0, -25, 0]]) - dictionary with values in mm and degrees (e.g. {"x": 600, "y": 0, "z": 300, "roll": 0, ...})
velocity	Float or List of Floats	The velocity of a simple path can be given as a single floating-point number or a list of numbers (in degrees per second). If a single value is given, this value is used as the maximum velocity for all path segments. If a list is entered, the given velocity values correspond to the maximum velocity in each segment of the path.
acceleration	Float or List of Floats	The acceleration of a simple path can be given as a single number or a list of numbers (in degrees per second squared). If a single acceleration is given, this value is used as the maximum acceleration for all path segments. If a list is entered, the given acceleration values correspond to the maximum acceleration in each segment of the path.

Examples

```
run_simple_path([[750, 0, 400, [180, 0, 0]],  
                [900, 0, 410, [170, 10, 10]],  
                [700, 100, 410, [170, 10, 10]]], velocity=100, acceleration=50)
```

This example runs a simple path through the given three path poses. Due to the interpolated movements, the robot will only stop at the end of the path, but not in intermediate poses. The velocity and acceleration values of 100°/s and 50°/s², respectively, guarantees that the robot tries to move as fast as possible, but not exceeding these values at any time.

```
run_simple_path(["pose1", "pose2", "pose3", "pose4"],  
                velocity=[30, 60, 90], acceleration=[50, 100, 150])
```

This example runs a path through four poses that have been saved as "pose1", "pose2", "pose3" and "pose4". The robot will move slow at first and increase its speed with each new path segment.

```
run_simple_path([{"1": 20, "2": 20, "3": -20, "4": -20, "5": -20, "6": 20},  
                [-20, -20, 20, 20, 20, -20], "myPose"])
```

This example runs a path that combines poses of different formats.

Wait for Motor

```
wait_for_motor(actuator_ids=None)
```

This function waits for some joint or multiple joints of the robot to stop its or their movement.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
actuator_ids	Various	This argument contains the IDs of the motors to move. It can be - the ID of a single joint (e.g. 6), - a list of joint IDs for multiple joints (e.g. [1,4,6]), - the global constant ALL_KIN_JOINTS, which is equal to the list of all kinematic joints, - the global constant ALL_ACTUATORS, which is equal to the list of all actuators including end effectors.

Stop Motion

```
stop_motion()
```

This function stops and aborts the currently running motion. If no motion is running, this command is ignored.

Is Motor Moving

```
is_motor_moving(actuator_ids)
```

This function checks if one or multiple motors are currently moving.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
actuator_ids	Various	This argument contains the IDs of the motors to move. It can be - the ID of a single joint (e.g. 6), - a list of joint IDs for multiple joints (e.g. [1,4,6]), - the global constant ALL_KIN_JOINTS, which is equal to the list of all kinematic joints, - the global constant ALL_ACTUATORS, which is equal to the list of all actuators including end effectors.
return	Boolean or List of Booleans	If checking for a single motor, it returns True if that motor is moving and False otherwise. If checking for multiple motors, it returns a list of Booleans with each value corresponding to one motor.

Combined Examples

```
if any(is_motor_moving(ALL_KIN_JOINTS)):
    print("waiting for robot to finish movements")
    wait_for_motor(ALL_KIN_JOINTS)
```

Waiting for all joints of the robot to stop moving.

```
move_joint([1, 4, 6], [45, -90, 45], [45, 30, 60], [60, 45, 30], block=False)
wait_for_motor(6)
print("joint 6 finished its movement")
wait_for_motor([1, 4])
print("joints 1 & 4 did also finish their movement in the meantime")
```

Moving joints 1, 4 and 6 to different angles with different velocities and accelerations, hence the joint movement will not finish simultaneously. To register exactly when joint 6 stops moving, the movement is executed with the argument "block = False" and a "wait_for_motor" function is called thereafter.

```
move_to_pose("myPose", block=False)
wait(2)
stop_motion()
```

Starting a motion and stopping it after a timeout of two seconds.

```
move_joint([1, 4, 6], [45, -90, 45], [45, 30, 60], [60, 45, 30], block=False)
wait_for_motor(6)
stop_motion()
```

Starting a motion and stopping it immediately after joint 6 reached its target position.

```
move_joint([1, 4, 6], [45, -90, 45], [45, 30, 60], [60, 45, 30], block=False)
while any(is_motor_moving(ALL_KIN_JOINTS)):
    print("moving motors: " + str([i for i in ALL_KIN_JOINTS if is_motor_moving(i)]))
    wait(0.1)
```

Starting a motion and while moving printing the moving motors with a frequency of 10Hz.

2.2 Kinematic Functions

For details about the mathematical conventions and the coordinate systems available in myP, please refer to the [myP User Manual](#).

Read TCP Pose

```
read_tcp_pose()
```

This function reads the current pose of the tool center point (TCP).

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
return	List of Floats	Returns the TCP pose in the format [x, y, z, [roll, pitch, yaw]], where [x, y, z] (in mm) represents the position and [roll, pitch, yaw] (in degrees) the orientation of the TCP with respect to the base frame of the robot.

Example

```
tcp = read_tcp_pose()
[x, y, z] = tcp[:3]
orientation = tcp[3]
z += 100
move_tool(x, y, z, orientation)
# ... this is similar to ...
move_tool(0, 0, 100, [0, 0, 0], relative = True)
```

This example moves the TCP of the end-effector to a position located 100mm above the current end-effector position.

Get Grid Points

```
get_grid_points(grid_name)
```

This function returns a list of all the points of a predefined grid.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
grid_name	String	This argument contains the name of the grid (e.g. "myGrid") created with the grid editor.
return	List of Dictionaries	Returns the list of all the grid points of the specified grid. The grid points are returned in the order they are supposed to be iterated through and contain the position coordinates in the dictionary format {"x": ..., "y": ..., "z": ...}.

Example

```
move_to_pose("drop_pose")
open_gripper()
orientation = [180, -70, 0]
for point in get_grid_points("myGrid"):
    move_tool(point['x'], point['y'], point['z']+50, orientation)
    move_tool(point['x'], point['y'], point['z'], orientation)
    close_gripper()
    move_to_pose("drop_pose")
    open_gripper()
```

This example picks objects from a grid called "myGrid" and drops them at a specific pose called "drop_pose". For all poses, the orientation [180°, -70°, 0°] for roll, pitch, and yaw have been chosen.

Forward Kinematics

```
forward_kinematics(joint_angles)
```

This function uses the built-in forward kinematics to calculate the TCP from a given set of joint angles. The reverse of this function is the inverse kinematics.

Parameter	Types	Description
joint_angles	List of Floats	This argument contains the list of six joint angles (in degrees) according to the pose you want to calculate.
return	List of Floats	Returns the result in the format [x, y, z, [roll, pitch, yaw]], where [x, y, z] (in mm) represents the position and [roll, pitch, yaw] (in degrees) the orientation of the resulting pose with respect to the base frame of the robot.

Examples

```
object_pick_height = 40
tcp_pose = forward_kinematics([0, 110, 70, 0, 0, 0])
max_table_height = object_pick_height - tcp_pose[2]
print("maximum table height for picking this object: " + str(max_table_height))
```

If a robot is mounted on a table/platform and an object is to be picked from the floor, this example calculates the max. height of the table/platform in order to still reach the object.

```
tcp_pose = forward_kinematics(read_actuator_position(ALL_KIN_JOINTS))
# ... this is similar to ...
tcp_pose = read_tcp_pose()
```

Reading the current TCP pose in two ways, using the appropriate function and taking the detour via forward kinematics.

Inverse Kinematics

```
inverse_kinematics(x, y, z, orientation=None, previous_angles=None)
```

This function uses the built-in inverse kinematics to calculate one set of joint angles from a given set of TCP coordinates. Since there usually are multiple solutions, the inverse kinematics will return the solution that is "closest" to the set of angles given by the argument "previous_angles". The reverse of this function is the forward kinematics.

Parameter	Types	Description
x, y, z	Floats	These values define the three-dimensional position of the TCP (in mm) expressed in Cartesian coordinates in the world frame, the main coordinate system of the robot.
orientation	List of Floats	These values define the tool orientation (in degrees). The orientation is defined as [roll, pitch, yaw] .
previous_angles	List of Floats	If this argument is given as a list of six joint angles (in degrees), then the closest solution with respect to these previous angles is returned. If this argument is not given, the closest solution to the current set of joint angles is returned. Please note that this argument does only have an effect if the orientation is given and therefore the set of solutions is limited.
return	List of Floats	Returns the result as a list of six joint angles (in degrees).

Examples

```
joint_angles = inverse_kinematics(600, 0, 300)
tcp_pose = forward_kinematics(joint_angles)
orientation = tcp_pose[3]
print(orientation)
```

Using the inverse kinematics function without orientation argument, the automatically chosen orientation (e.g. of the move_tool(x, y, z) function) can be backtracked by applying the forward kinematics to its result.

```
inverse_kinematics(600, 0, 300, [180, -90, 0])
```

In order to check if a specific set of coordinates (in this case x=600, y=0, z=300, roll=180, pitch=-90 and yaw=0) returns a kinematic solution, the inverse kinematics can be used. In case of no solution, this function raises the appropriate error.

```
[x, y, z, orientation] = forward_kinematics([0, 0, 0, 0, 0, 0])
solution_1 = inverse_kinematics(x+100, y, z-200, orientation, [0, 90, -90, 0, 0, 0])
solution_2 = inverse_kinematics(x+100, y, z-200, orientation, [0, -90, 90, 0, 0, 0])
```

In order to get multiple solutions for the same pose, the previous angles statement can be changed accordingly. In this example, two solutions for a standing upright robot, slightly shifted to the front and downwards, are calculated.

Get Pose Angles

```
get_pose_angles(pose)
```

Retrieve joint angles of a pose.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
pose	String	Name of the Pose.
return	List of Floats	List of joint angles. The list is sorted starting with the joint 1 and ending with joint 6.

Example

```
# retrieve pose angles
angles = get_pose_angles("pose1")

# print the joint angle of joint 1
print(angles[0])
```

Print the joint angle of joint 1 in pose "pose1"

Get Pose Data

```
get_pose_data(pose)
```

Retrieve position and orientation of a pose.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
pose	String	Name of the Pose.
return	Dictionary	x, y, and z Coordinates and roll, pitch and yaw orientation angles.

Example

```
# retrieve pose angles
pose = get_pose_data("p1")

# print the pose angle of joint 1
print(pose["x"])
```

Print the x coordinate of pose "pose1"

Read IMU Data

```
read_imu_data()
```

Read the current measurements of the inertial measurement unit (IMU) that is integrated in the uppermost joint of P-Rob. (Note that old versions of P-Rob do not come with IMUs.)

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
return	List of Float	The current IMU measurement [x, y, z, rx, ry, rz], where the values (x, y, z) correspond to the linear acceleration (in meters per second squared) and the value (rx, ry, rz) correspond to the rotational velocity (in degrees per second).

2.3 Program Flow Scheduling

Wait

```
wait(seconds)
```

This function waits for time to pass before the next script command is executed.

Parameter	Types	Description
seconds	Float	This argument specifies how many seconds will be waited before the script continues its execution.

Examples

```
move_to_pose("pose1")
wait(3)
move_to_pose("pose2")
```

This example moves the robot to predefined poses and waits in-between the movements.

```
i = 0
while True:
    print(i)
    i += 1
    wait(0.1)
```

This example prints increasing numbers in an infinite while loop. A wait command controls the execution speed of this loop. Without waiting, the loop would execute as fast as the processor allows, which we strongly recommend to prevent in order to keep myP alive.

Run Script

```
run_script(script_name, arguments=None)
```

This function executes a previously saved script within the running script. The chosen script is executed entirely before the main script continues.

Parameter	Types	Description
script_name	String	This argument contains the name of the script to run (e.g. "myMainScript").
arguments	Diverse	The arguments variable contains the arguments passed down to this function. These arguments are readable from the sub-script by accessing the "script_arguments" variable.

Start Parallel Script

```
start_parallel_script(script_name)
```

This function starts the execution of a previously saved "parallel" script in the running script. The chosen script is executed in a parallel thread and the main script continues its execution.

Parameter	Types	Description
script_name	String	This argument contains the name of the script to run (e.g. "myParallelScript").

Combined Example

Let's consider the following three simple scripts that print their own progress:

```
# this is the code of the main script "main_1"
print("beginning of 1st main script")
start_parallel_script("parallel_1")
print("middle of 1st main script")
run_script("main_2")
print("end of 1st main script")

# this is the code of the parallel script "parallel_1"
print("beginning of parallel script")
wait(2)
print("middle of parallel script")
wait(2)
print("end of parallel script")

# this is the code of the main script "main_2"
print("beginning of 2nd main script")
wait(1.5)
print("middle of 2nd main script")
wait(1.5)
print("end of 2nd main script")
```

When executing "main_1", the program flow also executes the scripts "parallel_1" and "main_2". Once the script "main_2" starts, it is processed completely before returning the program flow to "main_1". The parallel script on the other hand starts its execution at a specific point in the program flow and runs in parallel until either the parallel script finishes or the main application reaches its end. From the application output below, it is clear that due to the timing of the applications, the parallel script did not reach the end of its program flow:

```
beginning of 1st main script
beginning of parallel script
middle of 1st main script
beginning of 2nd main script
middle of 2nd main script
middle of parallel script
end of 2nd main script
end of 1st main script
```

Stop Application

```
stop_application()
```

This function stops the currently running applications. This includes of course the selected main application, and any applications (e.g. other "main" or "parallel" applications) that have been started by this application as well. If a "background" script is running, this one keeps running.

Pause Application

```
pause_application()
```

This function pauses the currently running applications. This includes of course the selected main application, and any applications (e.g. other "main" or "parallel" applications) that have been started by this application as well. If a "background" script is running, this one keeps running.

Resume Application

```
resume_application()
```

This function resumes the currently pausing applications. Since myP is in paused state when this function applies, this function can only be called from "background" scripts.

Stop and Release

```
stop_and_release(actuator_ids=None)
```

This function stops running applications, the same way "stop_application" does, and then invokes directly the gravity compensation (release) mode.

Parameter	Types	Description
actuator_ids	Various	This argument contains the IDs of the joints to be released after stopping. It can be - the ID of a single joint (e.g. 6), - a list of joint IDs for multiple joints (e.g. [1,4,6]), - the global constant ALL_KIN_JOINTS, which is equal to the list of all kinematic joints, - the global constant ALL_ACTUATORS, which is equal to the list of all actuators including end effectors.

Combined Example

```
while True:
    command = dialog_text("waiting for command")
    if command == "stop":
        stop_application()
    elif command == "pause":
        pause_application()
    elif command == "release":
        stop_and_release()
    elif command == "continue":
        pass
    else:
        print("command not recognized")
    wait(0.5)
```

This example asks the user for a command input and executes different program flow functions according to the received command.

Is Status Ready

```
is_status_ready()
```

Function to check whether myP is idle and ready to receive commands. This function can only be used in a background application to monitor and react to status changes.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
return	Boolean	Returning true if the myP status is “Ready”, false otherwise.

Is Status Running

```
is_status_running()
```

Function to check whether a myP application (or calibration routine) is currently in progress. This function can only be used in a background application to monitor and react to status changes.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
return	Boolean	Returning true if the myP status is “Running” or “Calibrating”, false otherwise.

Is Status Paused

```
is_status_paused()
```

Function to check whether a myP application is currently pausing and waiting to be resumed (or aborted). This function can only be used in a background application to monitor and react to status changes.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
return	Boolean	Returning true if the myP status is “Paused”, false otherwise.

Is Status Busy

```
is_status_busy()
```

Function to check whether myP is currently busy. This state is similar to the running state, but the robot cannot be paused. This function can only be used in a background application to monitor and react to status changes.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
return	Boolean	Returning true if the myP status is “Released”, “Recording” or “Processing”, false otherwise.

2.4 User Communication

Dialog Functions

```
dialog_choice(text, options, title="", dialog_type=0, dialog_value=None)
dialog_dropdown(text, options, title="", dialog_type=0, dialog_value=None)
dialog_text(text, title="", dialog_type=0, dialog_value=None)
dialog_yes_no(text, title="", dialog_type=0, dialog_value=None)
```

These dialog functions are used to open a dialog window that requires the user to answer a question, choose an option, or press a button. The following dialog types are available:

- The **Choice Dialog** takes a list of options and displays these options as buttons for the user to press.
- The **Dropdown Dialog** takes a list of options and displays them in a dropdown list for the user to select and accept.
- The **Text Dialog** lets the user enter text.
- The **Yes-No Dialog** lets the user press on predefined “Yes” and “No” buttons.

Parameter	Types	Description
text	String	The text displayed in the dialog window.
options	List of Strings	The text displayed for the selectable options.
title	String	The title displayed in the header of the dialog window.
dialog_type	Integer	The dialog type specifies the symbol shown in the dialog window and is for esthetic purposes only. The behaviour of this function is independent of the chosen dialog type. - 0: the symbol is a blue information sign - 1: the symbol is a green check mark - 2: the symbol is an orange exclamation mark - 3: the symbol is a red cross
dialog_value	String or Boolean	This value will be returned by default if the dialog is not answered properly. This happens e.g. if the user closes the dialog or if no GUI is open to display the dialog.
return	String or Boolean	For Yes-No Dialogs, the answer is returned as a Boolean, for other dialogs the selected option or entered text as a String.

Examples

```
task = dialog_choice("Looks like I'm idle. Which task do you want me to perform?",
                    options=["Clean Dishwasher", "Serve Coffee", "Entertain Me"],
                    title="Ready For Action", dialog_type=1)

run_script(task)
```

This example asks the user to choose between some tasks. Upon choosing, the robot will execute an application with the same name, which contains that task.

```
name = dialog_text("Please enter your name:", "User Input", dialog_value="<NAME>")
print(name)
```

This example asks the user to enter her or his name. The name is then printed.


```

message = "Please enter your age:"
dialog_type = 0
while True:
    age = dialog_text(message, "User Input", dialog_type)
    if age.isdigit():
        age = int(age)
        break
    else:
        message = "'" + age + "' is not a proper age. " \
            "Please enter your age properly (as a positive integer):"
        dialog_type = 3
    wait(1)
print(age)

```

This example asks the user to enter her or his age. If the age is not entered as a positive integer, the user is asked to enter it properly and the symbol changes to a red cross due to remind the user that something went wrong. The age is then printed.

```

good_day = dialog_yes_no("Are you having a good day?", "Question", 1)
if good_day:
    print("Enjoy your good day!")
else:
    print("Don't worry, be happy!")

```

This example asks the user a simple question that can be answered with "Yes" or "No" and prints different messages according to the chosen answer.

```

while True:
    ...
    if error:
        error_check = dialog_yes_no("An error occurred! Do you want " \
            "to continue anyway?", "Custom Error", 2)

        if not error_check:
            break

```

This example executes some custom code (shown as "...") in an infinite while loop. If an error occurs, the user is asked to continue with the next iteration of this loop or to break out of the loop to exit the application.

Print

```
print(message)
```

This function prints a message into the Application Output section of the Application Menu.

Parameter	Types	Description
message	String	This argument contains the message to print.

Example

```
print("Hello world!")
```

This example prints "Hello world!" in the output section of the application menu.

Say

Unfortunately, this function no longer works as intended due to third-party package changes we have no control over! It still works if you are using the myP GUI via Google Chrome and if you're using the languages English or German.

```
say(phrase, language=None, gender="m", block=True)
```

This function sends a message to the browser, where it is converted into an audio file and executed. This function requires an active internet connection.

Parameter	Types	Description
phrase	String	This argument contains the message to say in the desired language (e.g. "Hello world!").
language	String	This argument contains the language code of the phrase to speak (e.g. "gb" for UK English, "de" for German, "fr" for French or "cn" for Chinese/Mandarin). The English language is chosen by default.
gender	String	This argument contains the gender of the voice that speaks. Some languages are available in male ("m"), some in female ("f") and some in both. If a gender is not available, the default voice will be used.
block	Boolean	If True , the next command will be executed once myP finishes saying the full message. If False , the next command will be executed immediately. Please note, that in previous versions of myP, this function was always non-blocking.

Example

```
say("Hello, how are you?", "gb")
wait(4)
say("Hallo, wie geht es?", "de")
wait(4)
say("Bonjour, comment allez-vous?", "fr")
wait(4)
say("Salve, come sta?", "it")
```

Saying "Hello, how are you?" in different languages.

List of available languages

Language	Code	Genders
Afrikaans	af	m
Albanian	sq	m
Arabic	ar	m
Armenian	hy	f
Australian	au	f
Brasilian Portuguese	br	f
Bosnian	bs	m
Catalan	catalonia	m
Chinese Mandarin	cn	f
Croatian	hr	m
Czech	cz	f / m
Danish	dk (f) & da (m)	
Dutch	nl	f
English (UK)	gb	f / m
English (US)	us	f / m
Esperanto	eo	m
Finnish	fi	f / m
French	fr	f
German	de	f
Greek	gr	f / m
Haitian Creole	ht	f
Hindi	hi	f
Hungarian	hu	f / m
Icelandic	is	m

Language	Code	Genders
Indonesian	id	f
Italian	it	f
Japanese	jp	f
Korean	kr	f
Latin	va	f / m
Latvian	lv	m
Macedonian	mk	m
Moldavian	md	m
Montenegrin	me	m
Norwegian	no	f / m
Polish	pl	f
Romanian	ro	m
Russian	ru	f
Serbian	sr	m
Serbo-Croatian	hr	m
Slovak	sk	f / m
Spanish	es	f
Swahili	sw	m
Swedish	sv	f / m
Tamil	hi	m
Thai	th	f
Turkish	tr	f
Vietnamese	vi	m
Welsh	cy	m

(Please note that this list of available options has been taken from the integrated text-to-speech service and may or may not be complete. Only a selection of language and gender combinations have been verified to work properly.)

Create TCP Client

```
create_tcp_client(ip, port)
```

Create a tcp client that can be used to both send and receive messages. The client will continuously try to connect to the server at the specified address. Messages are encoded with the **UTF-8** encoding standard. Furthermore, messages are separated by the **End of Text** character 0x03 (“\x03” in Python).

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
ip	String	IP of the server (e.g. “192.168.80.2”)
port	Integer	Port through which the server is accessed (e.g. 60001). Ports are limited between 60000 and 61000.
return	Socket	The client socket. View the socket section below for further information.

Example

```
# create a client that connects to ip 192.168.80.2 through port 60001
client = create_tcp_client("192.168.80.2", 60001)
# wait until a message was received from the server and print it
print(client.receive())
# send an answer to the server
client.send("I am your client")
# close the connection
client.close()
```

Create a client and wait until a message is received. Upon receiving a message, send an answer to the server and close the client.

Create TCP Server

```
create_tcp_server(port)
```

Create a tcp server that can be used to both send and receive messages. The server can handle up to 20 client connections. Messages are encoded with the **UTF-8** encoding standard. Furthermore, messages are separated by the **End of Text** character 0x03 (“\x03” in Python).

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
port	Integer	Port through which the server can be accessed (e.g. 60002). Ports are limited between 60000 and 61000.
return	Socket	The server socket. View the socket section below for further information.

Example

```
# create a server that can be reached through port 60002
server = create_tcp_server(60002)
# wait until a message was received from any client and print it
print(server.receive())
# send an answer to all clients
server.send("I am your server")
```

```
# close the connection for all clients
server.close()
```

Create a server and wait until a message is received from any client. Upon receiving a message, send an answer to all clients and close the server.

Create UDP Socket

```
create_udp_socket(port)
```

Create a udp socket that can be used to both send and receive messages. Messages are encoded with the **UTF-8** encoding standard. Furthermore, messages are separated by the **End of Text** character 0x03 (“\x03” in Python).

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
port	Integer	Port through which the server can be accessed (e.g. 60002). Ports are limited between 60000 and 61000.
return	Socket	The udp socket. View the socket section below for further information.

Example

```
# create a server that can be reached through port 60003
udp_socket = create_udp_socket(60003)
# wait until a message was received from any client and print it
print(udp_socket.receive())
# send an answer to all clients
udp_socket.send("I am your socket")
# close the connection for all clients
udp_socket.close()
```

Create a socket and wait until a message is received from any peer. Upon receiving a message, send an answer to all peers and close the socket.

Send Message

```
send_message(message)
```

Send data from the script to all watchers. This data can be read by any watcher (for example by the TCP module).

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
message	various	The message that is sent to the watcher

Example

```
# send hello world string to watchers
send_message("hello world")
```

The hello message is broadcast to all watchers (e.g. TCP {"script_send": "hello world"})

Receive Message

```
receive_message(timeout=None)
```

Receive data from any watcher. If the timeout parameter is None, the function will wait until a message is received. If the function times out, it returns None instead of a value. Note that if a message was received prior to this function being called, said message is returned immediately.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
timeout	float or None	Time until the function times out (in seconds).
return	various	Message that was received from a watcher

Example

```
# wait up to 3 seconds for a new message
data = receive_message(timeout=3)
```

Wait until a message is received or three seconds have passed.

Clear Messages

```
clear_messages():
```

The received messages from a tcp watcher might pile up in the message queue. Call this function to make sure that any old message is cleared from the buffer if you are not interested in receiving them.

Example

```
# make sure there are no old boring messages
clear_messages()
# wait for a new exciting message
data = receive_message(timeout=3)
```

Socket

A socket is an object that is returned by the **create_tcp_client**, the **create_tcp_server** and the **create_tcp_socket** function. The object contains the following functions.

Socket Receive

```
socket.receive(timeout=None)
```

Wait until a message is received from any peer.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
timeout	Float	Time until the command timeout in seconds. If timeout is None the function will wait until the script is stopped.
return	String	Receive message. Returns an empty string if the function timed out.

Example

```
# wait until a message was received from any client and print it
print(socket.receive(timeout=3.0))
```

Receive and print a message. The function times out after 3 seconds.

Socket Send

```
socket.send(message)
```

Send a message to all connected peers.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
message	String	Message that is to be sent.

Example

```
# send "hello world" to all connected peers
socket.send("hello world")
```

Send a “hello world” to all connected peers.

Socket Close

```
socket.close()
```

Close this connection.

Example

```
# close the connection
socket.close()
```

Close the connection.

2.5 Custom Error Handling

Raise Error

```
raise_error(message)
```

This function raises a custom error (of type "ApplicationError") and aborts the running application. This error is shown in the user interface and also logged into the "error.log" file. This log file can be downloaded with all other myP log files by pressing "CTRL + SHIFT + L".

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
message	String	Error message to display and log into the "error.log" file.

Raise Warning

```
raise_warning(message)
```

This function raises a custom warning and displays it in a fading window in the user interface.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
message	String	Warning message to display.

Log Message

```
log_message(message, logger="custom", include_timestamp=True)
```

This function logs a custom message to an integrated custom log file. This log file can be downloaded with all other myP log files by pressing "CTRL + SHIFT + L". We recommend to use this function only for error handling and debugging purposes and use the function "print()" instead.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
message	String	Message to log into the specified log file.
logger	String	Name of the logfile (default: "custom.log").
include_timestamp	Boolean	Specifies whether a timestamp is to be logged together with the message.

Clear Log

```
clear_log()
```

This function removes all custom log files.

Combined Example

```
temperature_high = False
while True:
    temperature = read_sensor_data("my_temperature_sensor")
    log_message("Temperature: " + str(temperature))
    if temperature is None:
        raise_error("Could not read temperature sensor.")
    elif not temperature_high and temperature > 60:
        temperature_high = True
        raise_warning("The temperature is getting high. The system may fail soon.")
    elif temperature_high and temperature < 55:
        temperature_high = False
    elif temperature > 85:
        raise_warning("The temperature is too high. Aborting the program now.")
        stop_application()
    wait(5)
```

This example reads and logs a temperature value in an interval of 5 seconds. An error is raised if the sensor doesn't provide valid data. Warnings are raised if the measured temperature is getting too high. Finally, if the temperature reaches a critical limit, the application is stopped.

2.6 Application Variable Functions

myP applications support three variable types that are used in three different use cases:

- **Shared Variables** exist only during run-time of a single application. Write shared variables to share values amongst multiple scripts running in parallel or sequence in that application.
- **Global Variables** share values not only between the scripts of an application, but between all applications and still exist even after rebooting the robot.
- **State Variables** are used in the Panel Interface, where they are very useful to e.g. monitor the state and progress of an application or to specify certain input values to an application. For more detailed information about State Variables, how to define, modify, and use them, please refer to the **myP User Manual**.

The following functions for shared, global, and state variables have a very similar layout, therefore we combine their explanation:

Set Variable

```
set_shared_variable(name, value)
set_global_variable(name, value)
set_state_variable(name, value)
```

These functions set/write the value of a shared, global, or state variable, given the name and new value of that variable.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
name	String	This argument contains the name of the variable to set.
value	Various	This argument contains the value of the variable to set. Please note that state variables have been defined with a specific type, so you will only be able to write state variables if the type matches.

Get Variable

```
get_shared_variable(name=None)
get_global_variable(name=None)
get_state_variable(name=None)
```

These functions read a specific variable, given its name. If a name is not provided, all variables of that type are read.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
name	String or None	This argument contains the name of the shared or global variable to return.
return	Various	This function returns the previously saved value of the referred variable. If no name is given, a dictionary containing all variables (names and values) is returned.

Remove Variable

```
remove_shared_variable(name)
remove_global_variable(name)
```

These functions remove a variable permanently, given its name. Please note that state variables cannot be removed this way, since they are defined within the application itself.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
name	String	This argument contains the name of the variable to remove.

Shared Variable Example

Let's consider the following two scripts:

```
# this is the main application
set_shared_variable("sensor_data", None)
set_shared_variable("terminate_parallel_script", False)
start_parallel_script("reading_sensor_data")
...
for i in range(10):
    print(get_shared_variable("sensor_data"))
    wait(3)
set_shared_variable("terminate_parallel_script", True)
...

# this is the parallel script "reading_sensor_data"
while not get_shared_variable("terminate_parallel_script"):
    data = read_sensor_data("my_sensor")
    if data is not None:
        set_shared_variable("sensor_data", data)
    wait(1)
remove_shared_variable("sensor_data")
remove_shared_variable("terminate_parallel_script")
```

By executing the **main application**, shared variables are used to transmit data between this script and the script "reading_sensor_data" that is running in parallel to the main script. The variable "sensor_data" is used to store the latest data read from a sensor and the variable "terminate_parallel_script" contains a flag that makes sure that the program flow stops reading the sensor once its data is no longer required. Removing the shared variables is in this case not specifically required, since they are deleted anyways after the end of the main script is reached. If the program runs for a long time though, it is recommended to remove unnecessary variables, especially when using global variables.

2.7 Digital I/O Functions

Read Digital Inputs/Outputs

```
read_digital_inputs(bits=None)
read_digital_outputs(bits=None)
```

These functions read either one, multiple or all of the 8-bit digital inputs and outputs of P-Rob. Note that both the inputs and outputs can be read, but only the digital outputs can be written.

Parameter	Types	Description
bits	Various	This argument specifies which bits to read. If specific bits are addressed, integers between 1 and 8 need to be given, referring to the numbers of these digital inputs, which are also visible at the front of P-Rob. Using different argument formats, you may choose to - read all bits (by default or by using None), - read a single bit (giving an integer, e.g. 3), - read multiple bits (giving a list of integers, e.g. [1, 4, 5]), - read multiple bits (giving a set of integers, e.g. {7, 8}).
return	Various	The return format is different for each possible type of the input argument "bits". It can be - an integer between 0 and 255 (if "bits" is None), - a bool (if "bits" is an integer), - a list of bools (if "bits" is a list of integers), - a dictionary of bools (if "bits" is a set of integers).

Example

```
all_inputs = read_digital_inputs()
print(all_inputs)
print(bin(all_inputs))
```

This example reads all digital inputs and prints them as an integer number in decimal form and binary form (e.g. 115 and 01110011, hence the digital inputs {1, 2, 5, 6, 7} are high and the digital inputs {3, 4, 8} are low).

```
print(read_digital_inputs(3))
```

This example reads only the digital input 3 and prints its value (e.g. False, which means the digital input 3 is low).

```
print(read_digital_inputs([1, 4, 5]))
```

This example reads the digital inputs {1, 4, 5} and prints their values as a list of bools (e.g. [True, False, True], which means that the inputs 1 and 5 are high and the input 4 is low).

```
print(read_digital_outputs({7, 8}))
```

This example reads the digital outputs {7, 8} and prints their values as a dictionary of bools (e.g. {"7": True, "8": False}, which means that the output 7 is high and the output 8 is low).

Write Digital Outputs

```
write_digital_outputs(values, bits=None)
```

These functions write either one, multiple or all of the 8-bit digital outputs of P-Rob. Note that only the outputs can be written, but both the digital inputs and outputs can be read. The input arguments "values" and "bits" need to have compatible formats, given in the list below.

Parameter	Types	Description
values	Various	This argument specifies the value to write to the digital outputs. If specific bits are addressed, Boolean values need to be written, where True refers to a high output voltage and False refers to a low output voltage. Using different argument formats, you may choose to write - an integer between 0 and 255 (if "bits" is None), - a bool (if "bits" is an integer), - a list of bools (if "bits" is a list of integers), - a dictionary of bools (if "bits" is None).
bits	Various	This argument specifies which bits to write. If specific bits are addressed, integers between 1 and 8 need to be given, referring to the numbers of these digital outputs, which are also visible at the front of P-Rob. Using different argument formats, you may choose to - write all bits (by default or if "values" is an integer), - write a single bit (giving an integer, e.g. 3), - write multiple bits (giving a list of integers, e.g. [1, 4, 5]), - write multiple bits (giving a set of integers, e.g. {7, 8}).

Example

```
write_digital_outputs(115)
# ... this is similar to ...
write_digital_outputs(0b01110011)
```

This example writes the values 01110011 (or 115 in decimal format) to all digital outputs. Hence, the digital outputs {1, 2, 5, 6, 7} are high and the digital outputs {3, 4, 8} are low).

```
write_digital_outputs(False, 3)
```

This example writes the value False to the digital output 3, which means that the output 3 is set to low.

```
write_digital_outputs([True, False, True], [1, 4, 5])
```

This example writes the values {True, False, True} to the digital outputs {1, 4, 5}, which means that the outputs 1 and 5 are set to high and the output 4 is set to low.

```
write_digital_outputs({"7": True, "8": False})
```

This example writes the values {True, False} to the digital outputs {7, 8}, which means that the output 7 is set to high and the output 8 is set to low.

2.8 Reading Motor Data

Read Actuator Position/Velocity/Current

```
read_actuator_position(actuator_ids)
read_actuator_velocity(actuator_ids)
read_actuator_current(actuator_ids)
```

These functions read the position, velocity and current measurements of one or multiple joints.

Parameter	Types	Description
actuator_ids	Various	This argument contains the IDs of the joints of whose position should be measured. It can be - the ID of a single joint (e.g. 6), - a list of joint IDs for multiple joints (e.g. [1,4,6]), - the global constant ALL_KIN_JOINTS, which is equal to the list of all kinematic joints, - the global constant ALL_ACTUATORS, which is equal to the list of all actuators including end effectors.
return	List of Floats	Returns a list of position, velocity or current measurements (in degrees , degrees per second and amperes , respectively), one for each of the joints specified.

Examples

```
data = read_actuator_position(ALL_KIN_JOINTS)
print("joint angles of all kinematic joints: " + str(data))
```

This example prints the current angles of all kinematic joints.

```
data = read_actuator_velocity([1, 4, 6])
print("joint velocities of all wrist joints: " + str(data))
```

This example prints the current angles of all wrist joints (1, 4 and 6).

```
data = read_actuator_current(4)
print("current of joint 4: " + str(data))
```

This example prints the current velocities of joint 4.

2.9 Calibration Functions

Calibration functions are needed to calibrate the motors of the robot and its extended devices. These functions can be combined together with standard motor movement functions to create custom calibration scripts that can be executed instead of the default calibration procedure of P-Rob. Customized calibration procedures are especially important if the default calibration is not executable due to workspace restrictions.

Default Calibration

```
default_calibration()
```

This function performs the default calibration procedure that calibrates the robot including additional motors such as a connected end-effector. This function is usually used during a custom calibration script to either move the robot out of collision prone areas before the calibration, or to move the robot to a custom home position after the calibration procedure.

Calibrate Joint

```
calibrate_joint(actuator_ids, directions)
```

This function calibrates one joint of the robotic arm. The calibration moves this joint in either the positive or negative direction as far as it can move and then sets the zero value of this motor in the center of the joint range, such that the robot is standing up straight if all joints are calibrated correctly. Please make sure that the robot does not hit a wall, an object or itself during this procedure, otherwise the calibration of this joint will fail or result in wrong angular values for this joint.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
actuator_ids	String or Integer	This argument contains the joint ID of the joint to calibrate (e.g. 4).
directions	Integer	This argument contains the direction of the joint to calibrate. If the direction is a positive number, the joint will be calibrated in its positive rotation direction (e.g. -1 will cause a wrist joint to rotate around the negative z-axis or an elbow joint to rotate around the negative y-axis).

Custom Calibration Procedure Example

```
# 1st, calibrate the joints of P-Rob
calibrate_joint(1, 1)
calibrate_joint(4, -1)
calibrate_joint(6, 1)
move_joint([1, 4, 6], 0)
calibrate_joint(5, -1)
calibrate_joint(3, 1)
calibrate_joint(2, -1)
move_joint([2, 3, 5], 0)

# 2nd, calibrate a connected P-Grip
calibrate_gripper()

# intermediate step, move to a safe homing position before calibrating linear axes
move_joint([3, 5], [30, 60])

# 3rd, calibrate connected linear axes
calibrate_linear_axis(8, -1)
calibrate_linear_axis(9, -1)
```

This example shows a possible custom calibration procedure. Please note that this example will (very probably) not run on your system, but can be taken as a reference for your own calibration routine. This example combines calibration functions from various sections of this document.

Depending on the hardware setup you're facing, it may make sense to change the order in which motors are calibrated. Most importantly is making sure that the robot does not collide with obstacles during the calibration routine. Therefore, we usually suggest to the user to calibrate the available motors in the following order:

- P-Rob "wrist" motors
- P-Rob "elbow" motors
- P-Grip motor (if connected)
- Other external motors (such as e.g. Linear Axes)

2.10 Collision Detection

The collision sensitivity is an integer value between 0 and 10 and specifies how sensitive the robot (or single joints) react to collisions. In a robot with very low sensitivity, a lot of force needs to be applied for collisions to be triggered. In a robot with very high sensitivity, only little force needs to be applied for collisions to be triggered (and collisions may even be detected by accident without any external force applied). By default, all sensitivity values are set to 5.

Get Collision Sensitivity

```
get_collision_sensitivity()
```

This function returns the collision sensitivity values of all robot joints.

Parameter	Types	Description
return	Dictionary	This function returns the sensitivity values as integers between 0 and 10.

Set Collision Sensitivity

```
set_collision_sensitivity(actuator_ids, sensitivity)
```

This function modifies the collision sensitivity values of the robot joints.

Parameter	Types	Description
actuator_ids	Various	This argument contains the IDs of the joints whose position should be measured. Possible arguments are: - the ID of a single joint (e.g. 4) - a set of joint IDs for multiple joints (e.g. {"1", "2"})
sensitivity	Various	This argument contains the sensitivity values (between 0 and 10) for each joint to modify. Possible arguments are: - the sensitivity value for single joints (e.g. 7) - the dictionary of joint IDs and sensitivity values for multiple joints (e.g. {"1": 2, "2": 1})

Combined Example

```
# initial sensitivity values: {'1': 5, '2': 5, '3': 5, '4': 5, '5': 5, '6': 5}
set_collision_sensitivity({'1', '2'}, {'1': 2, '2': 1})
set_collision_sensitivity(4, 7)
print(get_collision_sensitivity())
```

This example makes the joints 1 and 2 insensitive and joint 4 more sensitive to collisions.

Get Collision Mode

```
get_collision_mode()
```

This function returns the currently active collision detection mode.

Parameter	Types	Description
return	String	This function returns the currently active collision detection mode as a string. The possible collision modes are: - "pause_and_resume" - "pause_and_approve" - "stop_and_error" - "stop_and_release" - "raise_interrupt" For details about the collision detection, please refer to the collision detection settings in the myP User Manual.

Set Collision Mode

```
set_collision_mode(new_mode)
```

This function changes the collision detection mode.

Parameter	Types	Description
new_mode	String	This argument contains the new collision detection mode to set. The possible collision modes are: - "pause_and_resume" - "pause_and_approve" - "stop_and_error" - "stop_and_release" - "raise_interrupt" For details about the collision detection, please refer to the collision detection settings in the myP User Manual.

Combined Example

```
mode = get_collision_mode()
set_collision_mode("stop_and_release")
play_path("test_path")
set_collision_mode(mode)
```

This example runs a path for testing purposes, for maximum security the collision mode is set to "stop_and_release" for this specific motion, and afterwards reset to its original value.

3 P-Grip Module Functions

Open Gripper

```
open_gripper(position=None, velocity=None, acceleration=None)
```

This function opens the gripper with optional arguments such as opening angle, velocity, or acceleration. By default, the gripper will open to its upper angular limit.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
position	Float	This argument contains the opening angle (in degrees) of the gripper (e.g. 15).
velocity	Float	This argument contains the angular velocity (in degrees per second) of the gripper (e.g. 30).
acceleration	Float	This argument contains the angular acceleration (in degrees per second squared) of the gripper (e.g. 40).

Examples

```
open_gripper()
```

Open the gripper to its upper angular limit using default values for velocity and acceleration.

```
open_gripper(15, 30, 40)
```

Open the gripper to an opening angle of 15° by moving with high velocity and acceleration values of 30°/s and 40°/s², respectively.

Close Gripper

```
close_gripper(velocity=None, acceleration=None, current=None)
```

This function closes the gripper with optional arguments for position, velocity, and acceleration. By default, the gripper will close until either its lower angular limit is reached or an object is grabbed.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
velocity	Float	This argument contains the angular velocity (in degrees per second) of the gripper (e.g. 30).
acceleration	Float	This argument contains the angular acceleration (in degrees per second squared) of the gripper (e.g. 40).
current	Float	This argument specifies the current (in amperes) that is applied on the gripper to hold an object (e.g. 0.2).

Examples

```
close_gripper()
```

Close the gripper by using default values for velocity and acceleration to grab an object.

```
close_gripper(30, 40)
```

Close the gripper and try to grab an object by moving the fingers with a velocity of 30°/s and an acceleration of 40°/s².

```
close_gripper(current=0.2)
```

Grab an object (if there is one) and hold it with a gripper current of 0.2A.

Read Gripper Angle

```
read_gripper_angle()
```

This function reads the current gripper angle.

Parameter	Types	Description
return	Float	Returns the current opening angle of the gripper (in degrees). Due to the elasticity of the fingers, this value might even be slightly negative if pressing the fingers together.

Example

```
close_gripper()
angle = read_gripper_angle()
if angle >= 10:
    print("big object has been grabbed")
elif 0 < angle < 10:
    print("small object has been grabbed")
else:
    print("no object has been grabbed")
```

By reading the gripper angle you can e.g. determine the size of an object. This example prints a message showing if a big or small object has been grabbed, by checking if the opening angle is bigger or smaller than 10°.

Calibrate Gripper

```
calibrate_gripper()
```

This function calibrates the motor of P-Grip. The calibration procedure closes the gripper with default force and takes the closed position as the reference angle of zero degrees. Please make sure not to place an object within the gripper fingers during calibration, otherwise the calibration may fail or result in wrong angular values.

For a detailed example of a custom calibration procedure including robot and gripper calibration, check out the [Calibration Functions](#).

Get Sensors

```
get_sensors()
```

This function returns a list of all available sensors in myP.

Parameter	Types	Description
return	List of Dictionaries	Returns a list of sensors dictionaries. Each dictionary contains the following data: - name (string) - type (string) - interval (float) - parameters (dictionary containing e.g. address, port)

Read Sensor Data

```
read_sensor_data(sensor_name)
```

This function reads the data of a sensor connected to the robot (e.g. through an open socket).

Parameter	Types	Description
sensor_name	String	This argument contains the name of the sensor (e.g. finger sensor “ finger_right_in ” or external sensor “ mySensor ”). The available internal and external sensors are shown in the sensor menu.
return	Various	Returns the data of a sensor. The data format is different for different types of sensors and depends on how this type of sensor is implemented in myP (e.g. finger sensors return raw integer values, the distance sensor returns the distance in cm). For further information on the gripper and finger sensors, please refer to the P-Grip User Manual).

Combined Examples

```
open_gripper()
distance_open = read_sensor_data("gripper_distance")
close_gripper()
distance_close = read_sensor_data("gripper_distance")
```

This example reads the distance sensor of P-Grip in open and closed gripper position.

```
for sensor_name in get_sensors():
    print(sensor_name + ": " + str(read_sensor_data(sensor_name)))
```

This example prints the names and values of all available sensors.

4 Linear Axis Module Functions

The following functions are only available if you installed the **Linear Axis Module**.

Move Linear Axis

```
move_linear_axis(actuator_ids, position, velocity=None, acceleration=None, jerk=None,
                 block=True, relative=False)
```

This function moves a linear axis to a specific position. This movement is smooth using an S-Curve velocity profile. This function is somewhat similar to "move_joint", but restricted in a way that it always uses S-Curves, cannot yet be used to move multiple axes simultaneously or start a motion with non-zero initial velocity. Also, certain combinations of velocity, acceleration and jerk arguments cannot be used for safety reasons. Therefore, using this command requires a bit of fine-tuning to find the optimal motion parameters.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
actuator_ids	Integer	This argument contains the ID of the motor to move (e.g. 8).
position	Float	This argument contains the target position (in mm).
velocity	Float	This argument contains the linear velocity (in mm per second) that is used during the motion (e.g. 100). If no argument is given, a relatively slow default velocity is chosen.
acceleration	Float	This argument contains the linear acceleration (in mm per second squared) that is used during the motion (e.g. 200). If no argument is given, the acceleration is chosen automatically.
jerk	Float	This argument contains the maximum jerk (in mm per second cubed) that is used to switch between the different constant acceleration phases of the motion (e.g. 400). If no argument is given, the jerk is chosen automatically. The jerk is mathematically defined as the derivative of the acceleration.
block	Boolean	If True , the next command will be executed once this motion has finished. If False , the next command will be executed without waiting for this motion to finish.
relative	Boolean	If True , the position values are interpreted as a relative offset to the current joint angles. If False , the position values are interpreted as absolute joint angles.

Examples

```
move_linear_axis(8, 1120, 100, 200, 400)
```

Moving a linear axis to the position 1120mm with respect to the selected origin (which is usually at one end of the linear axis). The motion uses the velocity, acceleration and jerk parameters of 100mm/s, 200mm/s² and 400mm/s³, respectively.

```
move_linear_axis(8, 1120, block=False)
move_linear_axis(9, -800, block=True)
wait_for_motor(8)
```

Moving with two linear axes asynchronously and waiting for both axes to finish their target position.

Calibrate Linear Axis

```
calibrate_linear_axis(actuator_ids, direction)
```

This function calibrates a linear axis. The calibration moves this axis in either the positive or negative direction as far as it can move and then sets this as the zero value for this axis. If a robot is mounted on the linear axis, please make sure that it does not hit a wall or an object during this procedure, otherwise the calibration of this axis will fail or result in wrong values for this axis. It is therefore strongly suggested to calibrate the robot before calibrating a linear axis.

<i>Parameter</i>	<i>Types</i>	<i>Description</i>
actuator_ids	String or Integer	This argument contains the joint ID of the linear axis to calibrate (e.g. 8).
direction	Integer	This argument contains the direction of the axis to calibrate. If the direction is a positive or negative number, the axis will be calibrated in its positive or negative movement direction, respectively.

For a detailed example of a custom calibration procedure including this function, check out the [Calibration Functions](#).

5 ROS Handler Functions

This module provides the user with a set of functions to handle ROS topics and messages in a myP application. If these functions are not sufficient for your application, you can always import the "rospy" Python package which contains the complete Python API for ROS.

All ROS handler related functions are called using the prefix `ros_handler.`

Publish to ROS Topic

```
ros_handler.publish_to_rostopic(topic_name, message)
```

Publishing a ROS message to a ROS topic.

Parameter	Types	Description
topic_name	String	Name of the ROS topic to publish to (e.g. "/myP/my_custom_topic"). If publishing to a topic that does not exist yet, it creates a ROS publisher.
message	Diverse	ROS message to publish. If publishing to an existing ROS topic, the type of this message must fit the message type given by the publisher. If publishing to a new ROS topic, a publisher is created, therefore you can choose the type to be basically anything.

Example

```
application_time = 0
while True:
    ros_handler.publish_to_rostopic("/myP/application_time", application_time)
    wait(0.5)
```

Publishing the up-time of a running application with a frequency of 2Hz.

Subscribe to ROS Topic

```
ros_handler.subscribe_to_rostopic(topic_name, callback=None, wait_for_data=False)
```

Function to subscribe to a ROS topic and react on incoming messages using a callback function.

Parameter	Types	Description
topic_name	String	Name of the ROS topic to subscribe to (e.g. "/myP/my_custom_topic").
callback	Function	This argument contains a function callback that is executed whenever the ROS topic specified gets new incoming data.
wait_for_data	Boolean	If True, this function call will wait for incoming data before continuing with the next command. This is useful if you want an application to wait for other devices to actively publish data or if you don't want to care about the order in which you start your ROS nodes.

Example

```
def subscriber_callback(message):
    print(message)
ros_handler.subscribe_to_rostopic("/movement_commands", subscriber_callback)
```

This example subscribes to a ROS topic that publishes sensors on some other robot in the same network and prints all incoming messages using a callback function.

```
command = None
def subscriber_callback(message):
    command = message["command"]
    if command == "stop":
        stop_application()
    elif command == "pause":
        pause_application()
    else:
        print("undefined command received: " + str(command))
ros_handler.subscribe_to_rostopic("/movement_commands", subscriber_callback,
                                wait_for_data=True)
print("first command received, controller is up and running, let's continue")
```

This example subscribes to a ROS topic that receives commands to stop or pause the running application coming from some other device in the same ROS network. For this to run properly, those commands could be sent in a ROS message of the format {"command": String}.

Read ROS Topic

```
ros_handler.read_rostopic(topic_name)
```

This function reads the newest ROS message from a ROS topic you previously subscribed to using the function "subscribe_to_rostopic".

Parameter	Types	Description
topic_name	String	Name of the ROS topic to read from (e.g. "/myP/my_custom_topic").
return	Diverse	Returns the newest ROS message on this ROS topic.

Example

```
ros_handler.subscribe_to_rostopic("/movement_commands", wait_for_data=True)
while True:
    command = ros_handler.read_rostopic("/movement_commands")
    if command == "stop":
        stop_application()
    elif command == "pause":
        pause_application()
    else:
        print("undefined command received: " + str(command))
    wait(5.0)
```

Compared to the previous example, this code uses active function calls to read commands with a predefined cycle time, instead of passively receiving commands using a callback.

Unregister from ROS Topic

```
ros_handler.unregister_from_rostopic(topic_name=None)
```

Unregister from one or all ROS topics currently subscribed to.

Parameter	Types	Description
topic_name	String	Name of the ROS topic to unsubscribe from (e.g. "/myP/my_custom_topic"). If no name given, unsubscribe from all previously subscribed topics.

Wait for ROS Topic

```
ros_handler.wait_for_rostopic(topic_name, timeout=None)
```

Waiting for a ROS message to be published on a ROS topic you subscribed to, and returning the next incoming message.

Parameter	Types	Description
topic_name	Type	Name of the ROS topic to unsubscribe from (e.g. "/myP/my_custom_topic"). If no name given, unsubscribe from all previously subscribed topics.
timeout	Float	If specified, wait for a maximum number of seconds defined by this argument.
return	Various	Returns the next incoming ROS message on the ROS topic to wait for, or None if the timeout expires.

Example

```
ros_handler.subscribe_to_rostopic("/some_topic")
data = ros_handler.wait_for_rostopic("/some_topic")
# ... this is similar to ...
ros_handler.subscribe_to_rostopic("/some_topic", wait_for_data=True)
data = ros_handler.read_rostopic("/some_topic")
```

This example shows two ways to wait for incoming ROS message data.

Get Available ROS Services

```
ros_handler.get_available_ros_services()
```

Returns a list of the names of all available ROS services (as strings), including those advertised by myP.

Parameter	Types	Description
return	List of Strings	Names of all available ROS services.